

Ho Chi Minh City University of Technology (HCMUT) — Faculty of Computer Science and Engineering

CO3133 Deep Learning and Its Applications — Semester 261 — Assignment 2

Six-Class Emotion Classification

A TF-IDF baseline vs. fine-tuned DistilBERT and BERT on `dair-ai/emotion`

Group Goat

Võ Lê Thành Triệu (2453297) — presenter

Hoàng Trọng Huy Minh (2452743) · Nguyễn Huỳnh Nam Quốc (2453089)

Instructor: Lê Thành Sách

Contents

1. Problem and data
2. Method
3. Results
4. Controlled experiments
5. Compute cost
6. Error analysis
7. Live demo
8. Conclusion

Click a section to jump to it; Contents in the footer of every slide comes back here. Backup slides (per-class F1, curves, confusion matrices, all 12 runs) follow the last slide.

One sentence in, one of six emotions out

The task

"im updating my blog because i feel shitty" → sadness

"i felt anger when at the end of a telephone call" → anger

"i find myself in the odd position of feeling supportive of" → love

Real texts from the test set.

Why it is hard

- **Imbalance:** joy has 9.37× the texts of surprise
- **Close emotions:** joy vs. love, fear vs. surprise
- **Noisy labels:** same text, two labels

Metric: Macro-F1 (+ accuracy)

Always predicting *joy*: accuracy **0.35**, Macro-F1 **0.09** (test)

Question: does fine-tuning a Transformer beat a TF-IDF baseline, and at what cost?

The data: `dair-ai/emotion` at a glance

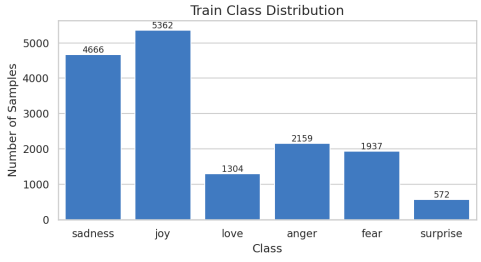
20,000
texts · 16k / 2k / 2k

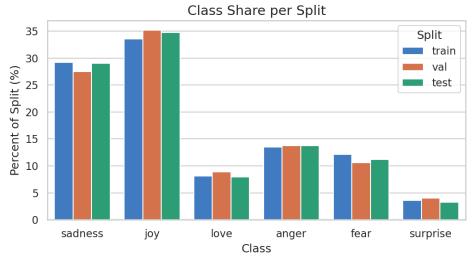
6
classes, one label per text

9.37×
joy vs. surprise (train)

17
median words per text

0
texts over 128 tokens (max 87)





Cleaning first, so nothing leaks into test



The four rules

1. Test stays untouched; its texts are removed from train and val
2. Text in train and val: drop the train copy
3. Same text with two labels: drop the whole group
4. Same text, same label: keep the first copy

Removed: 77 train, 7 val, 0 test. All 7 checks pass (no overlap, no repeats).

Two text versions

Minimal (every model): Unicode NFC, lowercase, single spaces

Aggressive (baseline variant): also drops 295 stopwords, **keeps** *not, no, never, but, very, so* ...

Vocabulary, IDF and class weights are fitted on train only.

Three models on one shared pipeline

BASELINE

TF-IDF + Logistic Regression

Word and word-pair counts, then a linear classifier

102,426 params · 1.2 MB
trains in 5.3 s on one CPU

MAIN MODEL

DistilBERT

6-layer Transformer, every weight fine-tuned

67.0 M params · 256 MB
trains in 277 s on a T4 GPU

COMPARISON

BERT

12-layer Transformer, the same recipe

109.5 M params · 418 MB
trains in 525 s on a T4 GPU

Added after the proposal: time remained, so we ran it to see what a larger model changes.



TF-IDF + LR: count features, then a linear classifier

A toy example: 3 texts, unigrams + bigrams

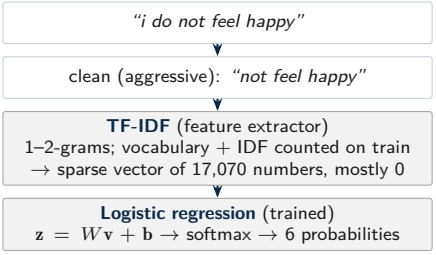
d1: feel happy today d2: **not feel happy** d3: feel scared today

n-gram in d2	in how many texts	IDF	weight
feel	3 of 3	1.00	0.32
happy	2 of 3	1.29	0.41
feel happy	2 of 3	1.29	0.41
not	1 of 3	1.69	**0.53**
not feel	1 of 3	1.69	**0.53**

$\text{IDF} = \ln \frac{1+n}{1+\text{df}} + 1$ weight = count × IDF, then scaled to length 1

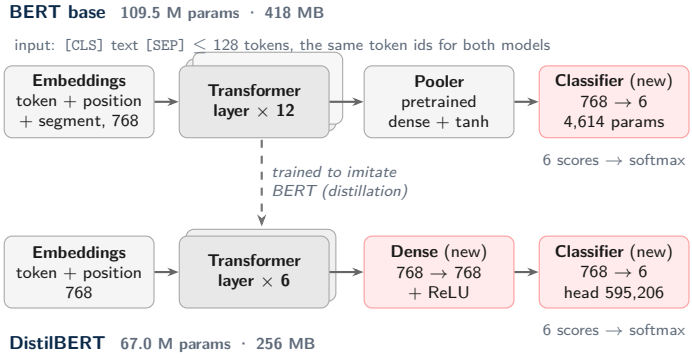
feel **is in every text → lowest weight;**

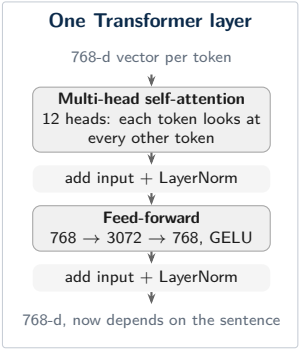
not **is rare → highest weight.**



TF-IDF is a fixed feature-extraction block: no gradient learning. Only the logistic regression is trained.

DistilBERT and BERT: same width, half the depth





pretrained, fine-tuned new, random start

Full fine-tuning updates every weight; the frozen variant trains the head only.

Training setup

Transformers (DistilBERT, BERT, frozen variant)

Loss	cross-entropy with class weights (love 2.07, surprise 4.71, joy 0.50)
Optimiser	AdamW, weight decay 0.01, gradient clip 1.0
Learning rate	encoder 2×10^{-5} · head 1×10^{-4}
Schedule	warmup 10% (298 steps), then linear decay to 0
Batch · epochs	16 · 3 (2,988 steps); max 128 tokens, per-batch padding
Checkpoint	the epoch with the best validation Macro-F1
Seeds	42, 43, 44 (42 = main run)

Baseline

Features	TF-IDF, 1–2-grams, min_df 2, max_df 0.9
Classifier	logistic regression, $C = 1$, L-BFGS
Weights	balanced (chosen in E4)
Text	aggressive (chosen in E3)

No hyperparameter search: common starting values, the same for every run.

Colab T4, FP32 · Python 3.13 · PyTorch 2.14.1 · transformers 5.22.1 · scikit-learn 1.6.1 · W&B logs

Final choice: DistilBERT, full fine-tune

Model	Val Macro-F1 (3 seeds, decides)	Test Macro-F1 (once, after)	Test Acc
Majority (joy)	0.0868	0.0860	0.3475
Frozen DistilBERT	0.4191 $\pm$0.0061	0.4199	0.5010
TF-IDF + LR	0.8758	0.8548	0.8905
DistilBERT full	**0.9155** $\pm$0.0020	**0.8902**	**0.9270**
BERT full	0.9163 $\pm$0.0032	0.8881	0.9280

Test columns = seed-42 run. The choice was fixed on validation before any test result.

Where context helps (test text, true label: fear)

*"i was feeling pretty anxious all day **but** my first day at work was a very good day and that helped a lot"*

DistilBERT: fear ✓ TF-IDF baseline: joy ✗

Why DistilBERT?

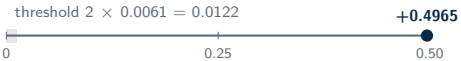
- ✓ **Beats the baseline:** +0.035 test Macro-F1, better on all 6 classes
- = **Ties with BERT:** gap +0.0008 is below the seed noise 0.0032 → keep the smaller model
- ✗ **Frozen encoder fails:** −0.50, even below the baseline

124 test texts only DistilBERT gets right vs. 51 only the baseline (McNemar $p \approx 3 \times 10^{-8}$)

How we decide: the gap vs. the seed noise

RULE A · REAL DIFFERENCE (E1)

A wins on **all 3 seeds** and the mean gap is $> 2 \times$ **the larger seed std**



full − frozen: about **40**× the threshold → real gap

RULE B · TIE (E2, FINAL CHOICE)

|mean gap| < **the larger seed std** → tie → keep the **smaller** model



BERT − DistilBERT: inside the noise band → tie

Why this heuristic?

1 Written down **before** the runs, as the handbook asks

2 Seed std = the noise from changing **only the seed**

3 ×2 (≈95% idea) + win on every seed = **conservative**

4 3 seeds: formal tests are weak, so a **simple rule** + McNemar per text

E1 (main): frozen encoder vs. full fine-tune

Hypothesis	pretrained features are not tuned to emotions → frozen is clearly worse, but trains faster
Changed	encoder frozen (595,206 trainable, 0.89%) vs. all 66,958,086 trained
Fixed	data, text, token ids, batch order, seeds, head LR, schedule, 3 epochs, class weights
Metric	val Macro-F1, mean ± std over 3 seeds
Criterion	(1) real gap = rule A (2) good enough = drop ≤ 0.02

0.4191
frozen, val · ± 0.0061

0.9155
full, val · ± 0.0020

- ✓ (1) full wins on 3/3 seeds; gap 0.4965 $\gg$ 0.0122
- ✗ (2) drop 0.50 $\gg$ 0.02 → **frozen excluded**
- ✓ test check: +0.4651, 3/3 seeds

Underfits: train = val loss 1.425, still falling when the LR reaches 0

Cost: 29% of the training time, but **not** faster at inference

Limit: one recipe only (same head LR, 3 epochs)

E2: does a bigger encoder help?

Hypothesis	the deeper BERT may gain a little at about twice the cost; is the gain above seed noise?
Changed	pretrained model: BERT vs. DistilBERT
Fixed	same token ids, data, recipe, batch order, seeds
Metric	val Macro-F1, mean $\pm$ std over 3 seeds, and seed by seed
Criterion	rule B: $\lvert\text{gap}\rvert <$ seed std $\rightarrow$ tie $\rightarrow$ smaller model

Limit: the swap bundles depth, BERT's pooler and distillation, so it shows **whether** BERT differs, not **why**.

BERT − DistilBERT, Macro-F1

Seed	Validation	Test
42	+0.0051	−0.0021
43	−0.0033	+0.0023
44	+0.0005	+0.0047
Mean	**+0.0008** $<$ 0.0032	**+0.0016** $<$ 0.0038

Tie on both splits → keep DistilBERT

McNemar (test): 62 vs. 60 texts, $p \approx 0.93$

Cost: $\times$1.64 params, $\times$1.89 train, $\times$2.00 CPU latency

E3 and E4: two cheap fixes for the baseline

E3 Stopwords removed vs. kept

Hypothesis	the signal is in emotion words, so little change
Changed	text: aggressive vs. minimal
Fixed	TF-IDF settings, $C = 1$, class weights
Metric	val Macro-F1
Rule	higher wins; a tie keeps the minimal text

+0.0269 val (0.8758 vs. 0.8489) · test +0.0183

F1 up on all 6 classes with half the n-grams; negated texts: accuracy 0.846 → 0.891

✗ **Rejected:** *not* now sits next to the emotion word → bigram *"not happy"*

E4 · ABLATION Class weights on vs. off

Hypothesis	small classes gain recall, lose some precision
Changed	class weights: balanced vs. none
Fixed	minimal text, TF-IDF settings, $C = 1$
Metric	val Macro-F1; per-class P / R / F1
Rule	higher wins; a tie keeps no weights

+0.096 val (0.8489 vs. 0.7527) · test +0.108

surprise recall 0.40 → 0.81, precision 0.89 → 0.73; F1 up on all 6 classes

✓ **Supported:** class weights used for every later model

Together they lift the baseline from 0.7289 to 0.8548 test Macro-F1.

Compute cost

Model	Trainable params	Size	Train time	Batched ms / text	One text (ms) CPU	One text (ms) GPU	Test Macro-F1
TF-IDF + LR	102,426	1.17 MB	5.3 s CPU	0.015 CPU	1.32	–	0.8548
DistilBERT, full	66,958,086	256 MB	276.9 ± 10.5 s	1.63	59.6	5.92	0.8861
DistilBERT, frozen	595,206	256 MB	81.4 ± 0.4 s	1.59	60.9	4.71	0.4210
BERT, full	**109,486,854**	**418 MB**	**524.7 ± 0.5 s**	**3.16**	**119.2**	**8.79**	0.8878

Transformers on a T4 GPU. Time: median of 5 repeats after a warm-up; one-text latency on the same CPU (1 thread). Train time and test Macro-F1: mean over 3 seeds.

BERT costs the most

×1.64 params, ×1.89 train time, ×2.00 CPU latency of DistilBERT, for **+0.0016** F1

Baseline is the cheapest

45× faster on CPU, 219× smaller than DistilBERT, for **−0.031** F1

Freezing saves training only

29% of the training time; same forward pass, same inference cost

Where do the 146 errors go?

How the pairs were chosen

1. Count every true → predicted pair (19 of the 30 possible pairs occur)
2. Rank by count: top 3 = **45%** of the errors, top 10 = **84%**
3. Merge a pair with its reverse and its neighbours: **3 groups**, one cause each
4. Also check the share of the class: surprise → fear is **15%** of all surprise texts

Every pair links two close emotions; none links opposites like joy ↔ anger.

True → predicted	●	●	●	% class	
joy → love	**42**	58	20	6.0	**G1**
anger → sadness	12	8	17	4.4	**G2**
anger → fear	12	7	7	4.4	**G2**
fear → surprise	11	19	26	4.9	**G3**
sadness → anger	11	23	5	1.9	**G2**
surprise → fear	10	2	0	**15.2**	**G3**
joy → surprise	7	11	7	1.0	**other**
love → joy	6	8	28	3.8	**G1**
sadness → fear	6	8	2	1.0	**G2**
fear → sadness	6	8	5	2.7	**G2**

Test errors: ● DistilBERT 146, ● baseline 219, ● BERT 144. % class = share of the true class (DistilBERT).

Three error groups, and a label-noise signal

Group	●	●	●	Likely cause	Possible fix
G1 joy ↔ love	48	66	48	shared words; love weight 2.07 vs. joy 0.50	more love data; per-class thresholds
G2 sadness/anger/fear	51	61	45	one negative word fits several emotions	multi-label; use more context
G3 fear ↔ surprise	21	21	26	smallest class; *overwhelmed*, *shocked*	more surprise data; threshold
other scattered	26	**71**	25	keyword errors of the baseline	–

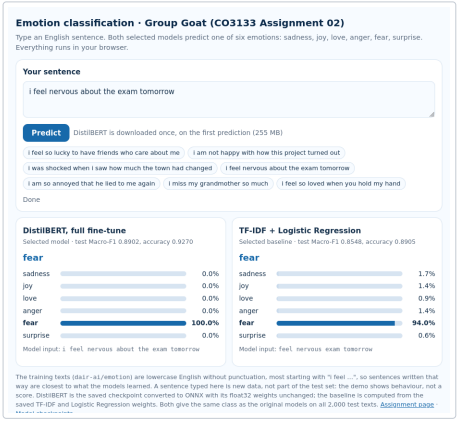
G1: *"...feel his force supporting me as i teach"* joy→love G2: *"when i learnt that my best friend had failed the exams"* anger→sadness G3: *"i started to feel so overwhelmed"* fear→surprise

50 test texts: all 3 models give the **same wrong** label (34% of DistilBERT errors)

9 / 9 disputed test texts: the error is the label of the removed copy

Transformers remove the scattered errors (71 → 26), not the close-emotion boundaries: the limit is the labels, not model size.

Live demo: try it in your browser



Open the live demo ↗

huggingface.co/spaces/TrieuVo/goat-a2-emotion-demo

- Our **DistilBERT** and **TF-IDF baseline**, side by side
- Runs fully in the browser (ONNX + transformers.js), no server
- Same class as our checkpoints on all 2,000 test texts

Try

i feel nervous about the exam tomorrow

i miss my grandmother so much

i am not happy with how this project turned out

Conclusion

0.8902 vs. 0.8548

Fine-tuning beats TF-IDF, on all 6 classes, most on long, negated and *"but"* texts

+0.0016 for ×2 cost

A bigger model does not help here: BERT ties with DistilBERT

0.42

The encoder must adapt: frozen DistilBERT is worse than the baseline

0.7289 → 0.8548

Cheap fixes matter: class weights + stopword removal; still 45× faster on CPU

Limits: noisy machine-generated labels · no hyperparameter search · only 3 seeds

Next: per-class thresholds · relabel doubtful texts · multi-label targets

Thank you

Questions are welcome.

Report page
volethanhtrieu.github.io/DL_Assignment-02

Code
github.com/volethanhtrieu/DL_Assignment-02

Live demo
huggingface.co/spaces/TrieuVo/goat-a2-emotion-demo

Checkpoints
huggingface.co/TrieuVo/goat-a2-emotion

Experiment logs
W&B project goat-a2-emotion

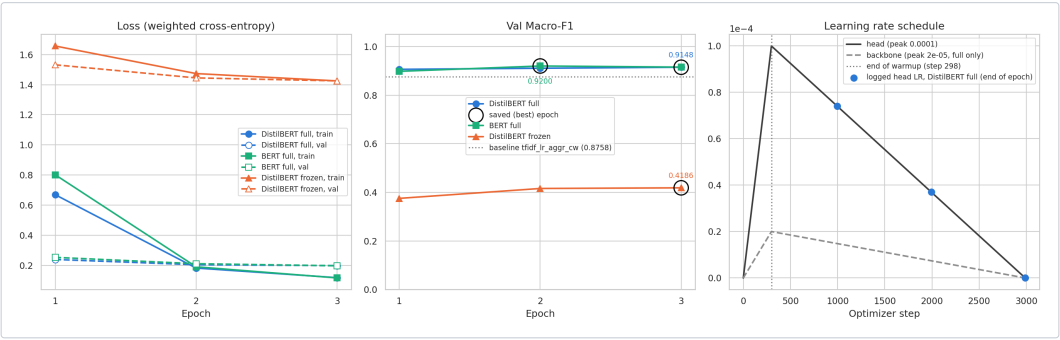
Presentation video
youtube.com/watch?v=ok-hF_vicjc

Per-class F1 on test (seed-42 runs)

Class	Test texts	Baseline	DistilBERT	BERT	Frozen	DistilBERT val
sadness	581	0.9296	0.9612	0.9643	0.5299	0.9616
joy	695	0.9110	0.9504	0.9513	0.6492	0.9561
love	159	0.7925	0.8523	0.8452	0.3435	0.8853
anger	275	0.8917	0.9134	0.9202	0.4329	0.9375
fear	224	0.8631	0.8904	0.8825	0.4219	0.8929
surprise	66	0.7407	0.7737	0.7654	0.1421	0.8554
Macro-F1	2,000	0.8548	**0.8902**	0.8881	0.4199	0.9148

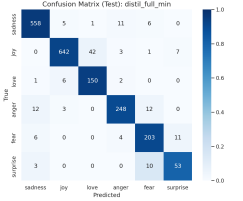
The val → test drop of the strong models comes mostly from surprise (DistilBERT −0.082, only 66 test texts) and love (−0.033).

Training curves (seed-42 runs)

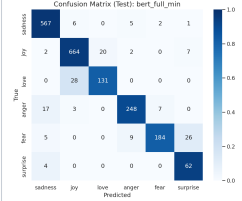


Left: loss (solid = train, dashed = val). Middle: val Macro-F1, saved epoch circled, dotted line = baseline. Right: learning rate per step. Full runs converge in 3 epochs; frozen runs underfit.

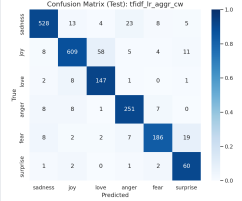
Confusion matrices on test (seed-42 runs)



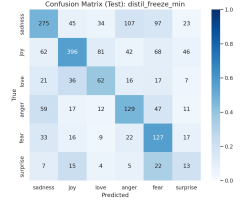
DistilBERT 0.8902



BERT 0.8881



Baseline 0.8548



Frozen 0.4199

All 12 runs on test

Model	Setup	Seed	Val F1	Test Acc	Test F1	Test − Val
DistilBERT	**full (selected)**	42	0.9148	0.9270	**0.8902**	−0.0246
DistilBERT	full	43	0.9178	0.9230	0.8854	−0.0324
DistilBERT	full	44	0.9140	0.9245	0.8828	−0.0313
BERT	full	42	0.9200	0.9280	0.8881	−0.0318
BERT	full	43	0.9145	0.9245	0.8877	−0.0268
BERT	full	44	0.9145	0.9270	0.8874	−0.0271
TF-IDF + LR	aggressive, weights (selected)	–	0.8758	0.8905	0.8548	−0.0210
TF-IDF + LR	minimal, weights	–	0.8489	0.8695	0.8365	−0.0124
TF-IDF + LR	minimal, no weights	–	0.7527	0.8325	0.7289	−0.0239
DistilBERT	frozen	42	0.4186	0.5010	0.4199	+0.0013
DistilBERT	frozen	43	0.4132	0.4940	0.4199	+0.0066
DistilBERT	frozen	44	0.4253	0.4970	0.4233	−0.0020
Majority (joy)	–	–	0.0868	0.3475	0.0860	–